

TP 4

Fédération de données multi-SGBD avec PySpark

Big Data Engineering — Master 1 — TP (2 h 30)

Prof. Samba Ndiaye • DMI/FST – UCAD
samba.ndiaye@ucad.edu.sn

Objectifs et livrable du TP 4

Au TP 3, Spark a joint deux tables de **deux bases MySQL**. Aujourd'hui : joindre **quatre tables** hébergées dans **quatre SGBD différents**.

- embarquer **plusieurs pilotes JDBC** dans une seule `SparkSession` ;
- lire des tables de 4 SGBD aux **dialectes** différents ;
- construire une **vue fédérée** par jointures multi-sources ;
- auditer l'**intégrité référentielle** inter-SGBD ;
- écrire le résultat consolidé dans un **entrepôt** (PostgreSQL).

[!] Livrable (à pousser sur GitHub avant la séance 5)

`notebooks/TP4_federation_multi_sgbd.ipynb` exécuté **avec ses sorties** + `docs/AUDIT_TP4.md` (les 3 anomalies d'intégrité + relevés).

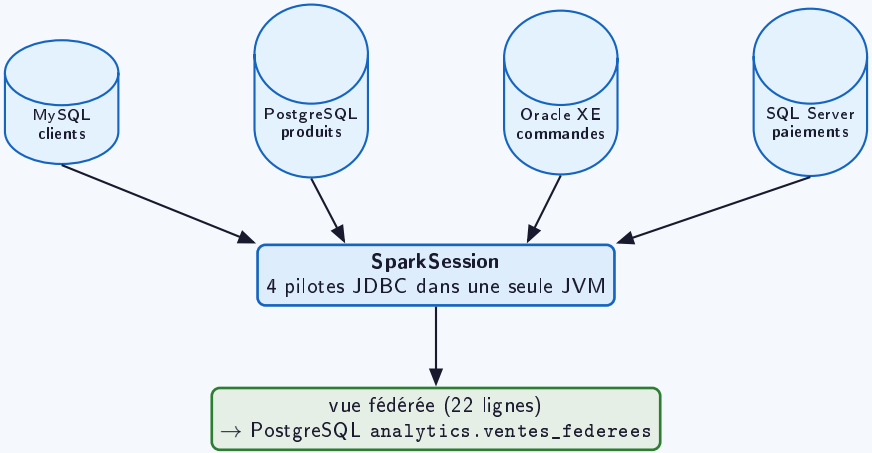
Le scénario : quatre équipes, quatre SGBD

Notre plateforme e-commerce sénégalaise a grandi. Chaque équipe a choisi **son** SGBD ; **aucune clé étrangère** ne traverse les systèmes.

Table	SGBD	Port	Base / schéma	Lignes
clients	MySQL 8	3306	`ecommerce_crm`	12
produits	PostgreSQL 16	5432	`ecommerce_catalogue`	10
commandes	Oracle XE 21c	1521	schéma `SPARK_USER`	30
paiements	SQL Server Express	1433	`ecommerce_paiements`	27

La direction veut un **CA consolidé** par ville, catégorie et moyen de paiement. Fil rouge
Compte technique commun : `spark_user` / `Ucad2026!` (moindre privilège, lecture seule).

L'architecture de fédération en image



Aucun SGBD ne sait interroger les trois autres. **Spark est le seul outil de la pile qui parle aux quatre à la fois.**

Déroulé de la séance (2 h 30)

Partie	Contenu	Durée
A	Mise en place des 4 SGBD (`init_tp4_*.sql`)	30 min
B	Les 4 connecteurs JDBC et la SparkSession	15 min
C	Lire les 4 tables (exercices 1–3)	25 min
D	La vue fédérée : jointures multi-SGBD (ex. 4–6)	30 min
E	Audit d'intégrité, analyses, entrepôt (ex. 7–9)	35 min
F	Discussion guidée et quiz éclair	15 min

Matériel fourni : le notebook étudiant + les 4 scripts `init_tp4_mysql.sql`, `init_tp4_postgres.sql`, `init_tp4_oracle.sql`, `init_tp4_sqlserver.sql`.

Rappel de cours : JDBC, le pont universel

[!] L'idée clé

JDBC (Java Database Connectivity) est une **interface unique** pour tout SGBD relationnel. Chaque éditeur fournit un **pilote** (un `.jar`) qui traduit cette interface vers son protocole natif.

- Spark lit une table via `spark.read.format("jdbc")` : il ne voit qu'un **DataFrame**, le dialecte reste caché dans le pilote.
- **Un pilote par SGBD** : MySQL, PostgreSQL, Oracle et SQL Server parlent des protocoles différents → **4 jars**.
- Le **schéma** (types des colonnes) vient du SGBD : fiable, sans `inferSchema` (contrairement au CSV du TP 2).

Rappel de cours : les dialectes divergent

Même donnée, quatre grammaires. Les pièges concrets du TP :

SGBD	Particularité	Conséquence côté Spark
MySQL	SSL exigé par défaut	suffixe `?useSSL=false...`
PostgreSQL	réf. propre	lecture directe, aucun piège
Oracle	schéma = utilisateur	table `spark_user.commandes`
Oracle	`NUMBER` générique	entiers en `decimal(38,10)`
Oracle	colonnes en MAJUSCULES	`toDF([c.lower()...])`
SQL Server	TCP/IP désactivé !	à activer avant tout (annexe)

/!\ À retenir

La fédération ne *supprime* pas les dialectes : elle les **réconcilie**. Une étape de **normalisation** (Oracle surtout) est presque toujours nécessaire.



PARTIE A

Mise en place des quatre SGBD

Partie A — exécuter les 4 scripts d'init

Chaque script (rejouable) crée la base/schéma, le compte `spark_user` et les données.
À lancer avec le **compte administrateur** du SGBD :

```
mysql  -u root -p < init_tp4_mysql.sql
psql   -U postgres -h localhost -f init_tp4_postgres.sql
sqlplus system/MdpSystem@//localhost:1521/XEPDB1 @init_tp4_oracle.sql
sqlcmd -S localhost -U sa -P MdpSa -i init_tp4_sqlserver.sql
```

Équivalents graphiques acceptés : Workbench, pgAdmin, SQL Developer, SSMS.
Vérifiez le décompte affiché en fin de script : **12 / 10 / 30 / 27**.

Partie A — pré-vérifier les 4 ports

La section 2 du notebook teste les 4 ports TCP **avant** Spark : un port fermé maintenant = une erreur JDBC illisible dans 10 minutes.

- la cellule doit afficher **quatre** `[ OK ]` ;
- chaque `[ECHEC]` imprime la marche à suivre pour ce SGBD.

/!\ Piège n°1 du TP — SQL Server Express

Le protocole **TCP/IP est désactivé par défaut**. Il faut l'activer (port 1433) *et* vérifier l'authentification en **mode mixte**. Détails dans l'annexe « pièges connus ».

PARTIE B

Quatre pilotes, une seule session

Partie B — charger les 4 pilotes JDBC

Le notebook télécharge les 4 jars dans `jars/` (contrôle de taille anti-fichier-corrompu du TP 3). Salle sans Internet : copiez-y les jars fournis, la cellule ne retéléchargera rien.

```
mysql-connector-j-8.4.0.jar    -> com.mysql.cj.jdbc.Driver
postgresql-42.7.3.jar          -> org.postgresql.Driver
ojdbc11-23.4.0.24.05.jar       -> oracle.jdbc.OracleDriver
mssql-jdbc-12.6.1.jre11.jar    -> com.microsoft.sqlserver.jdbc.SQLServerDriver
```

Partie B — une session, quatre jars

Nouveauté vs TP 3 : on passe les **4 jars d'un coup** à `spark.jars` (chaîne séparée par des virgules).

```
LISTE_JARS = ",".join(chemins)   # les 4 chemins absolus

spark = (SparkSession.builder
         .appName("TP4 - Federation multi-SGBD - UCAD")
         .master("local[*]")
         .config("spark.jars", LISTE_JARS)
         .getOrCreate())
```

/!\ Rappel du TP 3

`spark.jars` se fixe **à la création** de la session. Pour ajouter un jar ensuite, **redémarrez le noyau**.



PARTIE C

Lire les quatre tables

Partie C — un gabarit factorisé

Le gabarit JDBC est **identique** pour les 4 SGBD : seuls changent l'URL, la table et la classe du pilote. On factorise (réflexe des TP 2–3) :

```
def lire_jdbc(url, table, pilote, utilisateur=None, mdp=None):
    return (spark.read.format("jdbc")
            .option("url", url)
            .option("dbtable", table)
            .option("user", utilisateur or UTILISATEUR)
            .option("password", mdp or MDP)
            .option("driver", pilote)   # OBLIGATOIRE
            .load())
```

/!\ Piège n°2 — l'option `driver`

Sans elle : « No suitable driver », même si le jar est bien attaché. Vraie pour **les quatre** SGBD.

Partie C — exercices 1 à 3

MySQL (`clients`) est **guidé**. À vous les trois autres :

- **Ex. 1 — PostgreSQL** : lire `produits`, relever 10 lignes.
- **Ex. 2 — Oracle** : lire `spark_user.commandes` (30 lignes) *puis normaliser* le dialecte (voir ci-dessous).
- **Ex. 3 — SQL Server** : lire `dbo.paiements` (27 lignes).

```
# Ex. 2 : normaliser Oracle (colonnes MAJ + decimal -> types simples)
df_commandes = (df_commandes
    .toDF(*[c.lower() for c in df_commandes.columns])
    .withColumn("quantite",     F.col("quantite").cast("int"))
    .withColumn("montant_fcfa", F.col("montant_fcfa").cast("long")))
```

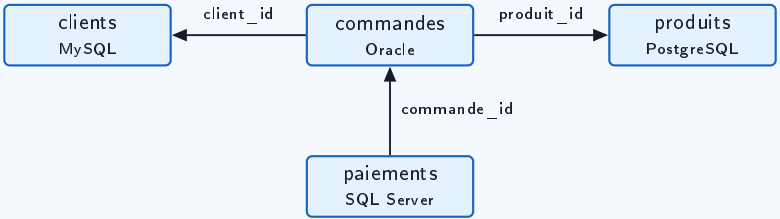


PARTIE D

La vue fédérée : jointures multi-SGBD

Partie D — le modèle logique (implicite)

Une fois les 4 DataFrames chargés, **leur origine ne compte plus** : Spark joint une table Oracle et une table MySQL comme deux CSV.



On expose chaque DataFrame en **vue temporaire** (`createOrReplaceTempView`), puis on écrit du **SQL standard** par-dessus — le *lingua franca* de la fédération.

Partie D — jointures progressives (ex. 4–6)

On monte de 2 à 4 SGBD. La table finale garde les commandes **livrées** au paiement **réussi** :

```
ventes_federees = spark.sql("""
    SELECT co.commande_id, cl.nom, cl.ville, pr.nom AS produit,
           pr.categorie, pa.moyen_paiement, co.montant_fcfa
    FROM commandes co
    JOIN clients   cl ON co.client_id   = cl.client_id
    JOIN produits  pr ON co.produit_id  = pr.produit_id
    JOIN paiements pa ON co.commande_id = pa.commande_id
    WHERE co.statut = 'livree' AND pa.statut_paiement = 'reussi'
""")
```

Partie D — pourquoi le compte diminue ?

Chaque jointure interne élimine les lignes sans correspondance :

Étape	Lignes	Cause
30 commandes	–	point de départ
+ clients (MySQL)	**29**	`CMD21` → client `C999` inconnu
+ produits (PostgreSQL)	**28**	`CMD22` → produit `P999` inconnu
+ paiements + filtres	**22**	livrées *et* payées avec succès

[!] La vue fédérée : 22 lignes

Chaque ligne mélange des données venues de **quatre SGBD différents**. **C'est la fédération de données.** Fil rouge



PARTIE E

Audit, analyses, entrepôt

Partie E — audit d'intégrité (ex. 7–8)

Aucune clé étrangère ne traverse les SGBD : **à l'ingénieur data de vérifier**. Outil : l'**anti-jointure** (`LEFT ANTI JOIN`).

```
# Paiements dont la commande n'existe PAS dans Oracle
paiements_orphelins = spark.sql("""
    SELECT pa.* FROM paiements pa
    LEFT ANTI JOIN commandes co ON pa.commande_id = co.commande_id
""")
```

Trois anomalies à trouver et documenter : `CMD21` (client fantôme), `CMD22` (produit fantôme), `PAY27` (paiement de 99 000 FCFA pour une commande `CMD99` **inconnue des Ventes**).

Partie E — un résultat vide est un résultat

Ex. 8 : lister les commandes **livrées mais sans paiement réussi** (candidates à la relance).

[!] Réponse attendue : 0 ligne

Dans ce jeu, toutes les livrées ont un paiement réussi. Un résultat **vide** *prouve* la cohérence livraison / encaissement — c'est un livrable d'audit à part entière, pas un échec de requête.

Cette anti-jointure sur une sous-requête (`paiements` réussis) est le motif dual de l'ex. 7 : on cherche l'**absence** de correspondance.

Partie E — analyses consolidées (ex. 9)

La vue fédérée en place, les questions métier deviennent du SQL ordinaire :

```
ca_ville = spark.sql("""
    SELECT ville, SUM(montant_fcfa) AS ca
    FROM ventes_federees GROUP BY ville ORDER BY ca DESC
""")
```

Regroupement	Tête de classement	CA (FCFA)
CA fédéré total	—	**1 452 000**
par ville	Dakar	723 000
par catégorie	Informatique	740 000
par moyen de paiement	Wave	537 000

Le mobile money (Wave + Orange Money) pèse ≈2/3 du CA — typique du e-commerce sénégalais. Exemple SN

Partie E — sous le capot : le pushdown

Spark ne rapatrie pas tout : quand c'est possible, il **pousse filtres et projections vers le SGBD** (*predicate / projection pushdown*).

```
(lire_jdbc(ORACLE_URL, "spark_user.commandes", ORACLE_PILOTE)
 .filter(F.col("STATUT") == "livree")
 .select("COMMANDE_ID", "MONTANT_FCFA")
 .explain())
# PushedFilters: [*IsNotNull(STATUT), *EqualTo(STATUT,livree)]
```

[!] Règle d'ingénieur

Filtrer **le plus tôt possible, le plus près possible de la donnée**. En production (des centaines de millions de lignes), sans pushdown Spark rapatrierait la table *entière* par le réseau avant de filtrer.

Partie E — boucler la boucle : l'entrepôt

Lecture fédérée → analyses → **écriture du résultat**. PostgreSQL joue l'entrepôt analytique (schéma `analytics`).

```
(ventes_federees.write.format("jdbc")
    .option("url", PG_URL)
    .option("dbtable", "analytics.ventes_federees")
    .option("user", UTILISATEUR).option("password", MDP)
    .option("driver", PG_PILOTE)
    .mode("overwrite").save())
```

On **relit** ensuite la table écrite (22 lignes) : preuve que la boucle est bouclée — 4 SGBD en entrée, 1 table consolidée en sortie, retrouvable dans pgAdmin/`psql`.



PARTIE F

Discussion guidée et quiz

Partie F — discussion guidée

En groupes de 3–4, restitution d'une minute par groupe :

1. La vue fédérée interroge les 4 bases de **production** à chaque exécution : quels **risques**, et quelle architecture du cours (séances 6–7) y répond ?
2. Qui, dans l'entreprise, devrait être **responsable** du paiement orphelin `PAY27` découvert par l'audit ?
3. Citez un cas **sénégalais réel** où la fédération multi-SGBD serait nécessaire (administrations, banques/fintech, opérateurs). Exemple SN

Partie F — quiz éclair (auto-évaluation)

1. Pourquoi faut-il **quatre** `.jar` différents ?
2. Que se passe-t-il si l'option `driver` est omise ?
3. Pourquoi la jointure interne perd-elle `CMD21` et `CMD22` ?
4. Qu'est-ce que le *predicate pushdown*, et **qui** exécute le filtre ?
5. `spark.jars` peut-il changer après création de la session ?

[!] Si vous hésitez sur une réponse

Relisez la partie correspondante du notebook : chaque piège y est commenté en « Observation attendue ».

Le livrable, en détail

À pousser sur GitHub **avant la séance 5** :

1. `notebooks/TP4_federation_multi_sgbd.ipynb` complété et exécuté de bout en bout, **avec ses sorties** ;
2. `docs/AUDIT_TP4.md` : les 3 anomalies (identifiants, table et SGBD source, impact métier en 1 phrase) + le tableau de relevés.

Critère	Poids
Notebook complet, exécuté sans erreur, relevés remplis	50 %
`AUDIT_TP4.md` : anomalies correctes et impact argumenté	30 %
Propreté du dépôt Git (structure, README, commits)	20 %

Avant de partir — checklist

- ☐ Les 4 scripts `init_tp4_*.sql` s'exécutent sans erreur
- ☐ La pré-vérification affiche **4** `[ OK ]`
- ☐ Vue fédérée = **22 lignes** ; CA total 1 452 000 FCFA
- ☐ Les **3 anomalies** d'audit sont identifiées et documentées
- ☐ `analytics.ventes_federees` visible dans pgAdmin/`psql`
- ☐ Notebook + `AUDIT_TP4.md` poussés sur GitHub

[!] Séance 5 — jointures avancées

Stratégies d'exécution : **broadcast** vs **shuffle** join. Relisez d'ici là la partie *pushdown* du notebook.

Annexe — pièges connus (1/2)

SQL Server Express — TCP/IP désactivé par défaut :

```
Configuration Manager -> Protocoles pour SQLEXPRESS -> TCP/IP -> Activer
  onglet Adresses IP -> IPAll : Port TCP = 1433
                              (vider "Ports dynamiques TCP")
Redemarrer le service SQL Server (SQLEXPRESS)
Verifier l'authentification en MODE MIXTE (sinon spark_user refuse)
```

Oracle XE : démarrer le *listener* (`...TNSListener`) **avant** `OracleServiceXE` ; le service de la base pluggable est `XEPDB1`, pas `XE`.

Annexe — pièges connus (2/2)

MySQL : conserver le suffixe d'URL (TP 3) :

```
jdbc:mysql://localhost:3306/ecommerce_crm?useSSL=false&\
    allowPublicKeyRetrieval=true
```

Tous SGBD : l'option `driver` est **obligatoire** dans chaque lecture — sans elle, « No suitable driver ».

Pare-feu Windows : si un port répond `[ECHEC]` alors que le service tourne, autorisez-le, ou testez avec :

```
Test-NetConnection localhost -Port 1433    # PowerShell
```



Université Cheikh Anta Diop de Dakar

DMI/FST – Prof. Samba Ndiaye

samba.ndiaye@ucad.edu.sn

Références principales : J. Reis & M. Housley, Fundamentals of Data Engineering (O'Reilly, 2022)
M. Kleppmann, Designing Data-Intensive Applications (O'Reilly, 2017)
J. Damji et al., Learning Spark, 2e éd. (O'Reilly, 2020)